MySQL 与 MongoDB 日志读写性能对比

温子健

摘　要：本文基于统一日志模型和 REST 接口，比较 Spring Data JPA/MySQL 与 Spring Data MongoDB/MongoDB 的日志读写性能。实验使用四台云主机分两轮部署，预置 10 万条日志，在 1 至 200 线程下测试查询与单条写入，每档重复 3 次。200 线程时，两库查询吞吐量分别为 1,258.0、1,167.8 次/秒，写入吞吐量分别为 4,608.5、4,236.2 次/秒。72 次正式测量共记录 20,730,914 次请求、0 次错误。结合响应时间分布与资源监测，分析增加并发的吞吐收益和延迟变化。

关键词：MySQL；MongoDB；Spring Data JPA；JMeter；Prometheus

Log Read and Write Performance of MySQL and MongoDB

CHIKIN WEN

Abstract: This experiment compares Spring Data JPA/MySQL and Spring Data MongoDB/MongoDB using a shared log model and REST API. Four cloud servers are reused in two sequential rounds. Each database starts with 100,000 records. Read and write workloads use 1 to 200 concurrent threads, with three repetitions per level. JMeter measures latency, throughput and errors, while Prometheus collects resource metrics every five seconds. At 200 threads, read throughput is 1258.0 requests/s for MySQL and 1167.8 requests/s for MongoDB; write throughput is 4608.5 and 4236.2 requests/s, respectively. The analysis relates latency distributions to application, database and load-generator resource usage.

Key words: MySQL; MongoDB; Spring Data JPA; JMeter; Prometheus

1　问题描述

日志服务需要持续写入事件，并按日志级别和时间范围查询历史记录。本文比较关系型表与文档集合在同一 Java 日志业务中的读写表现，重点考察并发增加时吞吐量、响应时间分布及资源占用的变化。两条链路使用相同接口、字段、请求生成规则和 JMeter 断言，使对照结果对应实际应用访问过程。

2　实验环境与方法

2.1　四机两轮部署

实验采用四台 Ubuntu 18.04 主机。第一轮运行 MySQL 和 JPA 应用，第二轮停止 MySQL 并切换为 MongoDB 及对应应用。两轮复用相同的数据库机、应用机、内网路径和资源配置，Compiler 的编译工作在压测开始前完成。

表 1　服务器配置与分工

主机	CPU 与可见内存	实验角色
Compiler	2 核，792 MiB	Java 17 编译与 JMeter 压测
Node1	2 核，1800 MiB	MySQL 或 MongoDB，分轮运行
Node2	2 核，792 MiB	当前被测 Spring 应用
Manager	2 核，1800 MiB	Prometheus 监控

表 2　软件与运行参数

项目	实际配置
应用	Spring Boot 3.3.13，Java 17.0.21，堆 256 MiB
编译与压测	Java 17.0.7，Maven 3.9.16，JMeter 5.6.3
MySQL	8.4.13，InnoDB 缓冲池 256 MiB
MongoDB	7.0.16，WiredTiger 缓存 0.25 GiB
数据库资源	两轮数据库容器内存上限均为 1 GiB
连接池	两条链路最大 20 个连接，最小 5 个
监控	Prometheus 3.7.3，node_exporter 1.10.2，5 秒采样

2.2　日志模型与统一接口

日志字段包括 id、traceId、level、source、message、clientIp 和 createdAt。id 采用 26 位 ULID，createdAt 统一保存 UTC 毫秒时间，message 最长 4096 字符。MySQL 以 CHAR(26)保存主键、DATETIME(6)保存时间；MongoDB 将 id 映射为字符串_id，以 BSON Date 保存时间。

POST /api/logs 执行单条写入并返回 HTTP 201 及保存的对象。GET /api/logs 接收 level、from、to、page、size，按左闭右开时间范围查询，返回 content、page、size、totalElements 和 totalPages。两套实现均按 createdAt 和 id 降序排列，并建立级别、时间、主键联合索引。

JPA 实现使用 EntityManager.persist 写入、JpaRepository 条件分页查询；MongoDB 实现使用 Spring Data MongoTemplate 执行 insert、count 和 find。两条读取链路均返回当前页与匹配总数。MongoTemplate 默认写入 _class 类型元数据，HTTP 业务字段保持统一。MySQL 每次事务提交刷盘并关闭二进制日志，MongoDB 采用单机 WiredTiger 及 w=1、journal=true 写入确认。

3　实验设计

3.1　基准数据与请求

生成器以 20261111 为随机种子创建 10 万条日志，时间均匀覆盖 2026 年 11 月 1 日 UTC 全天，包含五种级别和八个服务来源。两库导入同一批业务字段，导入后核对记录数。固定查询为 INFO 级别、UTC 06:00 至 07:00、第一页 20 条，匹配总数为 2497 条。写入请求使用 250 字符正文和固定业务时间，服务端生成 ULID。

3.2　先负载后压力的分析方法

负载测试先考察 1 和 10 线程下的查询与写入表现；逐级压力测试随后比较 1、10、25、50、100、200 线程，结合吞吐平台、P95/P99 增长、错误率与资源占用识别性能转折。JMeter 提供外部 HTTP 指标，Prometheus 提供主机和应用内部指标，两类指标共同用于上述测试。两章复用同一批阶梯并发测量，保留原运行时间顺序；正文按先负载、后压力组织分析。

查询与写入分别采用 1、10、25、50、100、200 线程，每档重复 3 次，两种数据库共 72 次正式测量。每次运行前恢复 10 万条基准数据，随后记录 10 秒升压、30 秒预热和 120 秒正式测量。正式窗口按请求开始时间截取；窗口结束附近的收尾请求单独保留，原始 HTTP 采样行可完整复核。预热写入计入本次数据库增长，下一次运行前恢复基准。写入测试将全程成功请求数与数据库记录增长量核对，查询测试核对记录总数保持不变。

3.3　指标与监控

JMeter 使用非 GUI 模式，保存完整 HTTP 响应时间及成功状态，不扣除网络连接耗时。统计每次运行的吞吐量、错误率、平均值和 P50/P95/P99；各档汇总采用 3 次测量的中位数，误差线展示最小值至最大值。吞吐量按正式窗口请求数除以 120 秒计算。Prometheus 同步记录主机 CPU、可用内存、磁盘读写、应用堆和 GC 指标，并监控压测机资源。

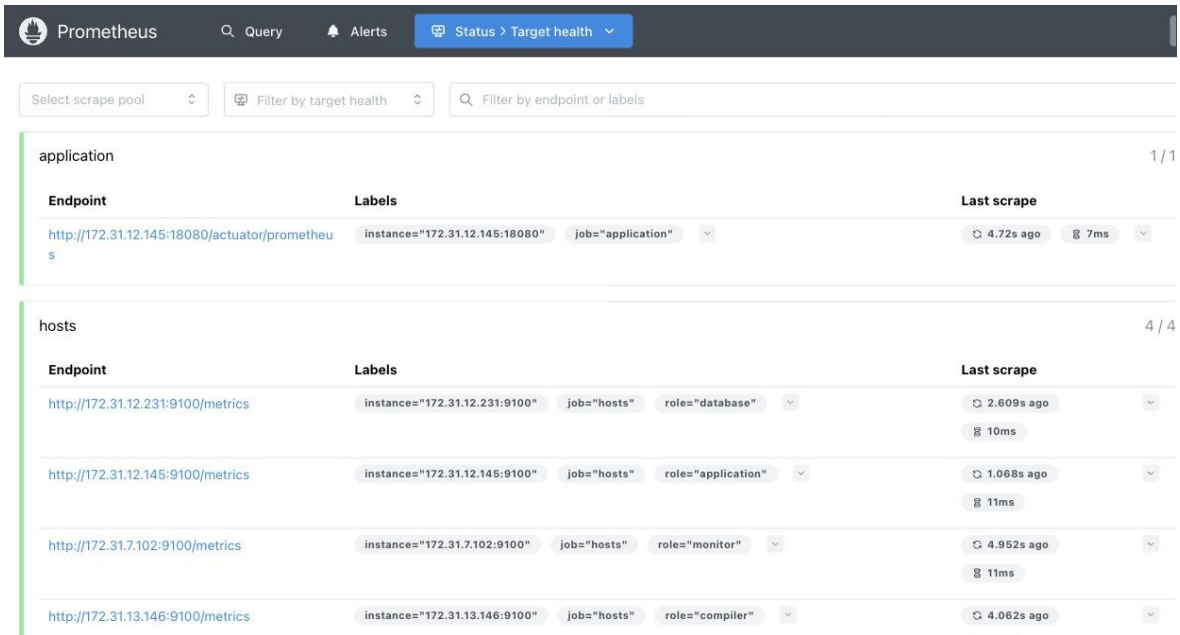


图 1　Prometheus 的应用与四台主机采集目标全部在线

3.4　功能与索引验收

两条实现分别通过 14 项 HTTP 检查，覆盖固定数据查询、分页互斥、空结果、负页码、页大小、非法级别、正文超长、UTC 毫秒精度及时间区间边界。固定查询的主键顺序与生成器计算的预期值一致。MySQL 的 EXPLAIN 和 MongoDB 的执行计划保存在 evidence 目录，用于核对联合索引的使用。

接口图为补拍的 Edge 开发者工具 Network 原图，依次展示 POST 状态码、请求正文、保存结果与 GET 分页预览。本机请求客户端通过 HTTP 转发和 SSH 隧道连接 Node2；接口验收的本地网络耗时与正式 JMeter 压测分别记录。两库各新增 1 条演示日志，固定查询仍匹配 2497 条。

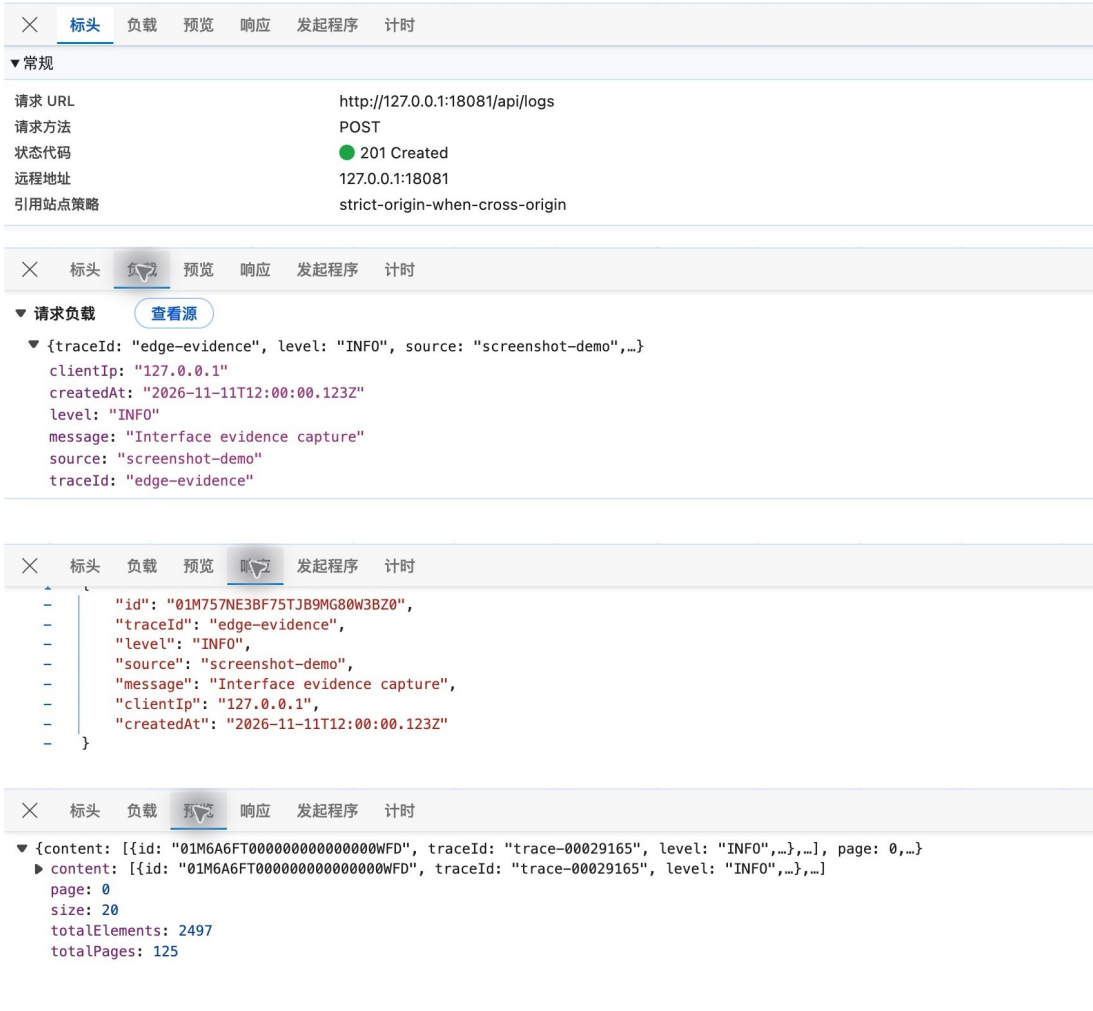


图 2　MySQL 读写接口验收

MongoDB 接口验收

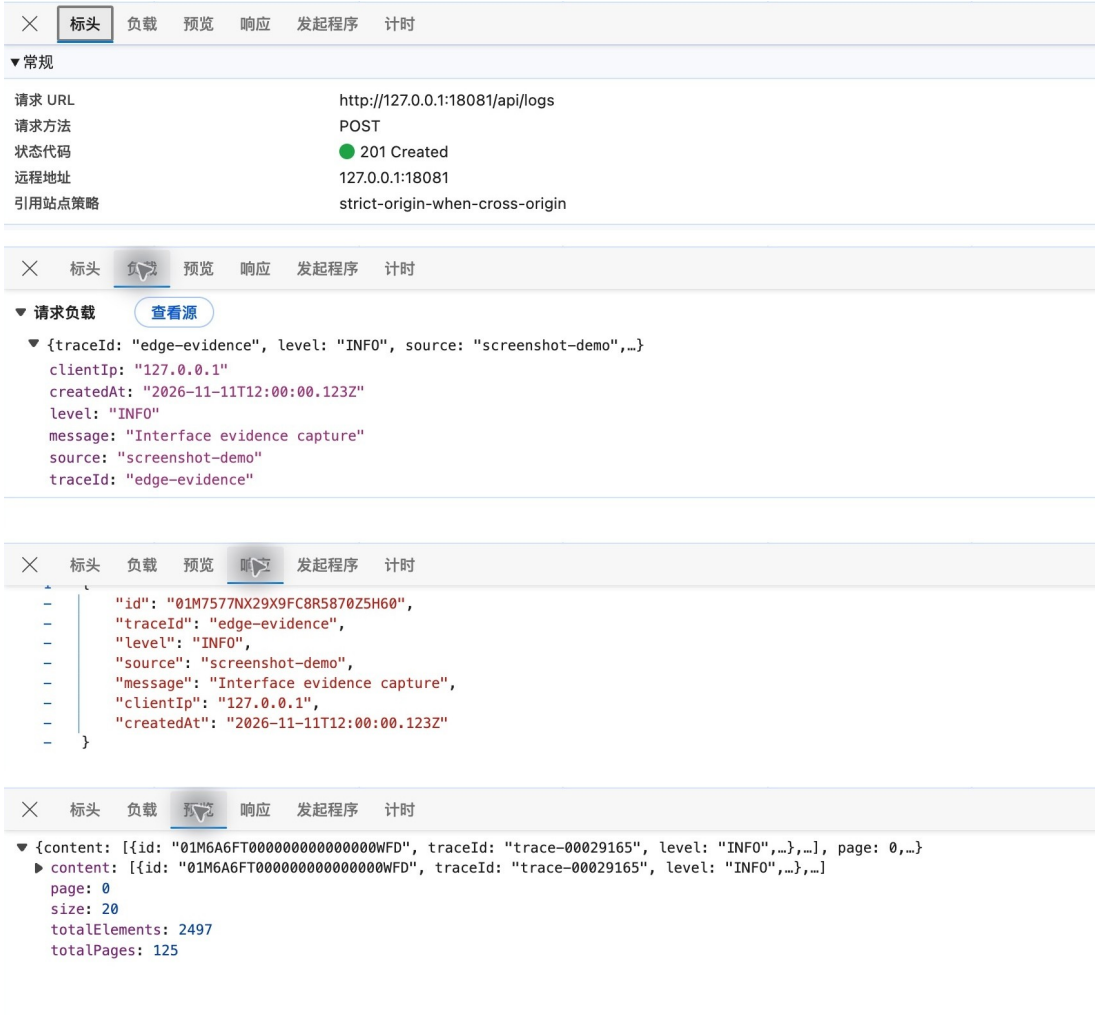


图 3　MongoDB 读写接口验收

3.5　数据库记录与索引

下图展示实际日志及当前记录数。正式末次写入测试结束后，MySQL 为 765118 条，MongoDB 为 766808 条；Edge 接口补拍各新增 1 条演示日志，因此截图总数分别为 765119 和 766809 条。MySQL 宽表按列分为上下两块，行序保持一致。

```
ssh -t root@119.3.221.88 'bash /opt/lab4/scripts/ghostty-evidence.sh mysql-records'
root@119.3.221.88's password:
Capture time: 2026-11-11 12:25:44 UTC
+-------------------+--------------------+
| current_log_count | evidence_demo_rows |
+-------------------+--------------------+
|            765119 |                  1 |
+-------------------+--------------------+
+----------------------------+----------------+-------+------------------+--------------------
| id                         | trace_id       | level | source           |
+----------------------------+----------------+-------+------------------+
| 01M6A6FT000000000000000WFD | trace-00029165 | INFO  | course-service-5 |
| 01M6A6FT000000000000000WFC | trace-00029164 | INFO  | course-service-4 |
+----------------------------+----------------+-------+------------------+
| message_excerpt                                                | client_ip    | created_at                 |
+----------------------------------------------------------------+--------------+----------------------------+
| Log event 00029165: course benchmark payload. course benchma | 10.0.165.210 | 2026-11-01 06:59:58.560000 |
| Log event 00029164: course benchmark payload. course benchma | 10.0.164.209 | 2026-11-01 06:59:57.696000 |
+----------------------------------------------------------------+--------------+----------------------------+
```

图 4　MySQL 存储记录与记录总数

```
ssh -t root@119.3.221.88 'bash /opt/lab4/scripts/ghostty-evidence.sh mongo-records'
root@119.3.221.88's password:
Capture time: 2026-11-11 12:26:28 UTC
{
  current_log_count: 766809,
  evidence_demo_rows: 1
}
[
  {
    _id: '01M6A6FT000000000000000WFD',
    traceId: 'trace-00029165',
    level: 'INFO',
    source: 'course-service-5',
    message: 'Log event 00029165: course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload.
 course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course
 benchmark payload. ',
    clientIp: '10.0.165.210',
    createdAt: ISODate('2026-11-01T06:59:58.560Z')
  },
  {
    _id: '01M6A6FT000000000000000WFC',
    traceId: 'trace-00029164',
    level: 'INFO',
    source: 'course-service-4',
    message: 'Log event 00029164: course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload.
 course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course benchmark payload. course
 benchmark payload. ',
    clientIp: '10.0.164.209',
    createdAt: ISODate('2026-11-01T06:59:57.696Z')
  }
]
```

图 5　MongoDB 存储记录与记录总数

联合索引与执行计划

MySQL 联合索引列为 level、created_at、id，执行计划使用 range 访问和 Using index condition，估计扫描 2497 行。MongoDB 联合索引列为 level、createdAt、_id，计划为 LIMIT→FETCH→IXSCAN；执行统计显示返回 20 条、检查 20 个索引键和 20 个文档。图 7 左侧为索引与扫描阶段，右侧接续索引属性、边界和执行统计。

```
ssh -t root@119.3.221.88 'bash /opt/lab4/scripts/ghostty-evidence.sh mysql-index'
root@119.3.221.88's password:
Capture time: 2026-11-11 12:30:00 UTC
+-------+------------+------------------+--------------+-------------+-----------+
| Table | Non_unique | Key_name         | Seq_in_index | Column_name | Collation |
+-------+------------+------------------+--------------+-------------+-----------+
| logs  |          0 | PRIMARY          |            1 | id          | A         |
| logs  |          1 | idx_level_time_id |            1 | level       | A         |
| logs  |          1 | idx_level_time_id |            2 | created_at  | D         |
| logs  |          1 | idx_level_time_id |            3 | id          | D         |
+-------+------------+------------------+--------------+-------------+-----------+

+----+-------------+-------+------------+-------+------------------+------------------+---------+------+------+----------+-----------------------+
| id | select_type | table | partitions | type  | possible_keys    | key              | key_len | ref  | rows | filtered | Extra                 |
+----+-------------+-------+------------+-------+------------------+------------------+---------+------+------+----------+-----------------------+
|  1 | SIMPLE      | logs  | NULL       | range | idx_level_time_id | idx_level_time_id | 50      | NULL | 2497 |   100.00 | Using index condition |
+----+-------------+-------+------------+-------+------------------+------------------+---------+------+------+----------+-----------------------+
```

图 6　MySQL 联合索引和查询计划

```
[
  {
    v: 2,
    key: {
      _id: 1
    },
    name: '_id_'
  },
  {
    v: 2,
    key: {
      level: 1,
      createdAt: -1,
      _id: -1
    },
    name: 'idx_level_time_id'
  }
]
{
  stage: 'LIMIT',
  limitAmount: 20,
  inputStage: {
    stage: 'FETCH',
    inputStage: {
      stage: 'IXSCAN',
      keyPattern: {
        level: 1,
        createdAt: -1,
        _id: -1
      },
      indexName: 'idx_level_time_id',
      isMultiKey: false,
      multiKeyPaths: {
        level: [],
        createdAt: [],
        _id: []
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        level: [
          '["INFO", "INFO"]'
        ],
        createdAt: [
          '(new Date(1793516400000), new Date(1793512800000)]'
        ],
        _id: [
          '[MaxKey, MinKey]'
        ]
      }
    }
  }
}
{
  nReturned: 20,
  totalKeysExamined: 20,
  totalDocsExamined: 20
}
```

图 7　MongoDB 联合索引和查询计划

4　负载测试结果与分析

本章先比较 1 和 10 线程下的基础负载表现。每档 3 次测量，表中列出中位数；各次吞吐量、平均值、P50/P95/P99 和错误率见 results/all-runs.csv。两类操作分别使用相同请求与数据库初始数据。

4.1　查询负载测试

表 3　查询基础负载性能

线程	MySQL 次/秒	P95 ms	P99 ms	MongoDB 次/秒	P95 ms	P99 ms
1	373.3	3.0	4.0	419.3	3.0	3.0
10	1198.1	12.0	15.0	1248.8	13.0	16.0

10 线程时，MySQL 吞吐量为 1,198.1 次/秒、P95 为 12.0 ms；MongoDB 为 1,248.8 次/秒、13.0 ms。两库 1 与 10 线程各 3 次正式测量的错误率均为 0%。

1 线程下，MySQL 的 P50/P95/P99 中位数为 3.0/3.0/4.0 ms；MongoDB 为 2.0/3.0/3.0 ms。下图为该档第 1 次重复的 JMeter 报告页面裁切图，左侧为 5 秒窗口的响应时间趋势，右侧为完整正式窗口的百分位分布。

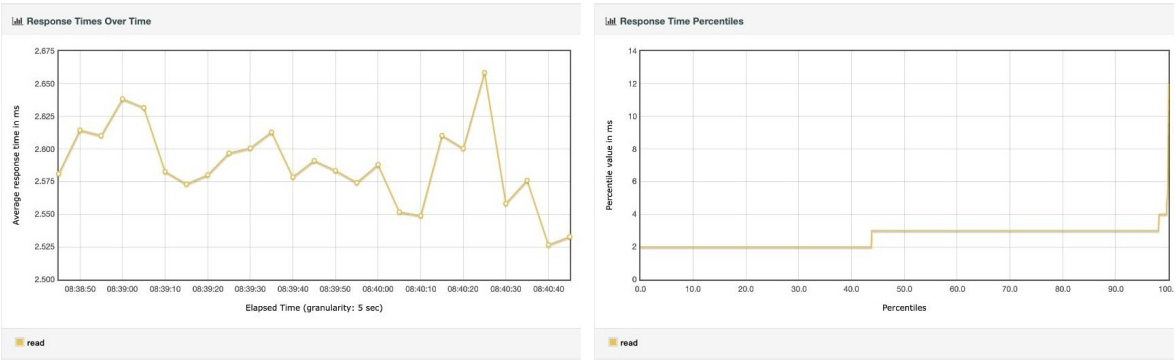


图 8　MySQL 查询 1 线程的趋势与百分位分布

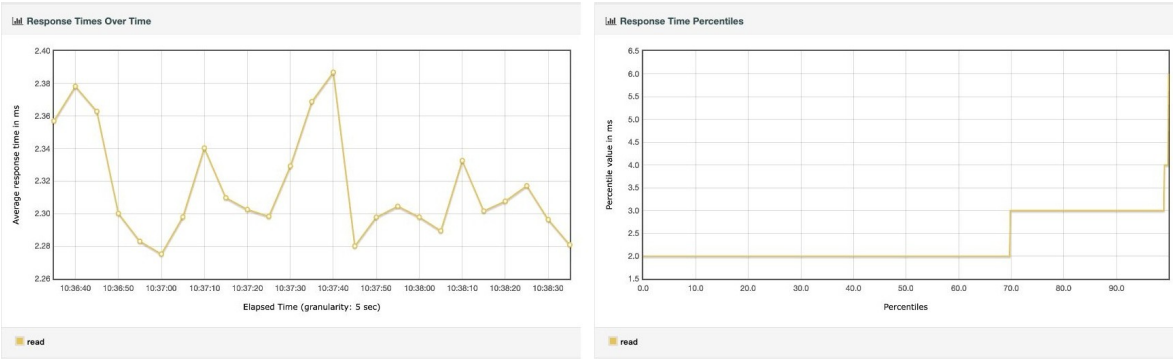


图 9　MongoDB 查询 1 线程的趋势与百分位分布

基础负载下的曲线用于观察短时波动与响应时间分布；后续压力测试使用全部并发档位，比较增加线程后的吞吐收益与延迟代价。

4.2　写入负载测试

表 4　写入基础负载性能

线程	MySQL 次/秒	P95 ms	P99 ms	MongoDB 次/秒	P95 ms	P99 ms
1	608.6	2.0	3.0	689.3	3.0	3.0
10	3909.9	4.0	5.0	3214.2	4.0	6.0

10 线程时，MySQL 吞吐量为 3,909.9 次/秒、P95 为 4.0 ms；MongoDB 为 3,214.2 次/秒、4.0 ms。两库 1 与 10 线程各 3 次正式测量的错误率均为 0%。

1 线程下，MySQL 的 P50/P95/P99 中位数为 2.0/2.0/3.0 ms；MongoDB 为 1.0/3.0/3.0 ms。下图为该档第 1 次重复的 JMeter 报告页面裁切图，左侧为 5 秒窗口的响应时间趋势，右侧为完整正式窗口的百分位分布。

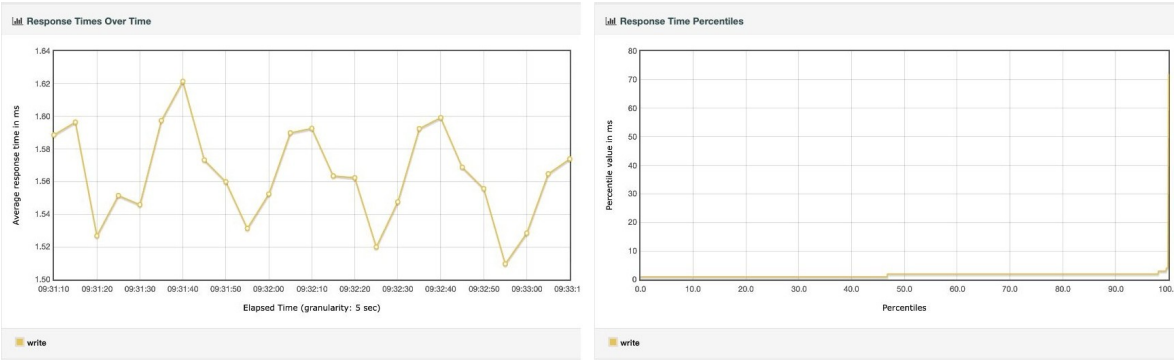


图 10　MySQL 写入 1 线程的趋势与百分位分布

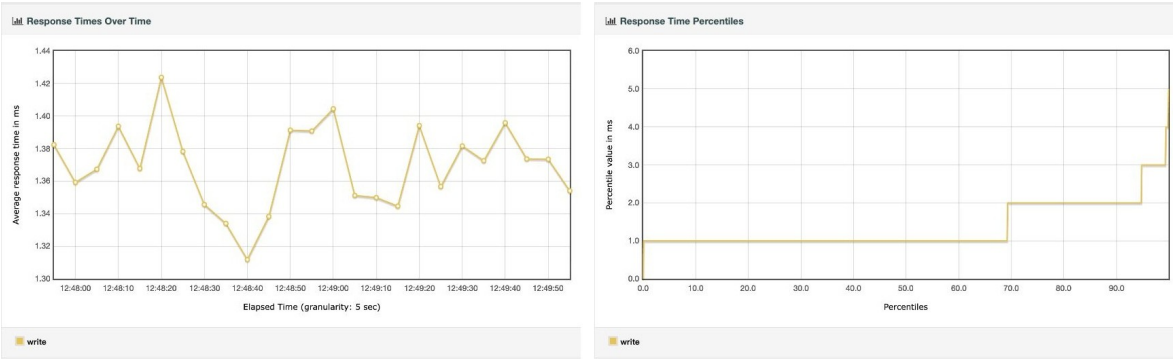


图 11　MongoDB 写入 1 线程的趋势与百分位分布

基础负载下的曲线用于观察短时波动与响应时间分布；后续压力测试使用全部并发档位，比较增加线程后的吞吐收益与延迟代价。

5　逐级压力测试结果与分析

本章比较 1 至 200 线程的阶梯升压结果。判断性能转折时同时检查吞吐量、延迟和错误率：增加线程后吞吐量趋平或回落、P95/P99 继续上升，表明额外并发主要增加了等待。全部档位均完成，正式请求错误数为 0；以下结论对应已测范围内的吞吐平台与延迟退化，故障上限和过载恢复未纳入本次测量。

5.1　查询阶梯升压

表 5　查询逐级压力测试性能

线程	MySQL 次/秒	P95 ms	P99 ms	MongoDB 次/秒	P95 ms	P99 ms
1	373.3	3.0	4.0	419.3	3.0	3.0
10	1198.1	12.0	15.0	1248.8	13.0	16.0
25	1304.7	30.0	36.0	1271.2	32.0	38.0
50	1307.4	73.0	109.0	1270.1	57.0	67.0
100	1285.8	145.0	210.0	1249.7	122.0	149.0
200	1258.0	305.0	444.0	1167.8	258.0	328.0

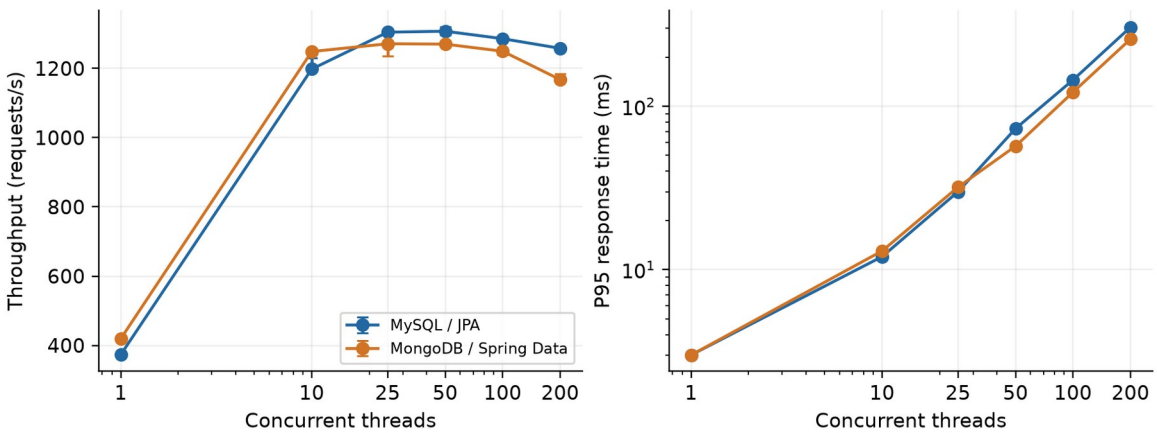


图 12　查询吞吐量及 P95 随并发变化，误差线为 3 次测量范围

在已测档位中，MySQL 最高中位吞吐量为 1,307.4 次/秒，对应 50 线程、P95 为 73.0 ms；MongoDB 最高中位吞吐量为 1,271.2 次/秒，对应 25 线程、P95 为 32.0 ms。

MySQL 从 50 线程增加到 200 线程，吞吐量较已测峰值下降 3.8%，P95 由 73.0 ms 上升至 305.0 ms，约为 4.2 倍。此区间继续增加并发的主要代价是延迟增长。

MongoDB 从 25 线程增加到 200 线程，吞吐量较已测峰值下降 8.1%，P95 由 32.0 ms 上升至 258.0 ms，约为 8.1 倍。此区间继续增加并发的主要代价是延迟增长。

在 200 线程档，MongoDB 相对 MySQL 的吞吐量低 7.2%、P95 低 15.4%。

查询 25 线程的 JMeter 曲线

25 线程下，MySQL 的 P50/P95/P99 中位数为 18.0/30.0/36.0 ms；MongoDB 为 19.0/32.0/38.0 ms。下图为该档第 1 次重复的 JMeter 报告页面裁切图，左侧为 5 秒窗口的响应时间趋势，右侧为完整正式窗口的百分位分布。

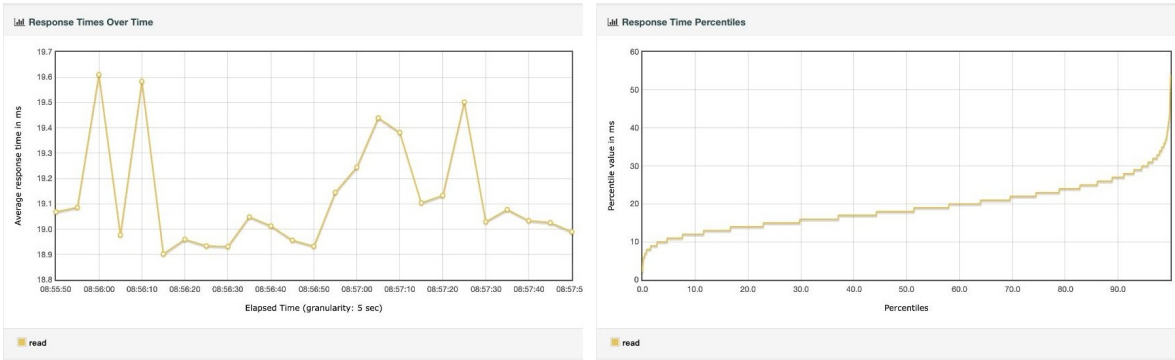


图 13　MySQL 查询 25 线程的趋势与百分位分布

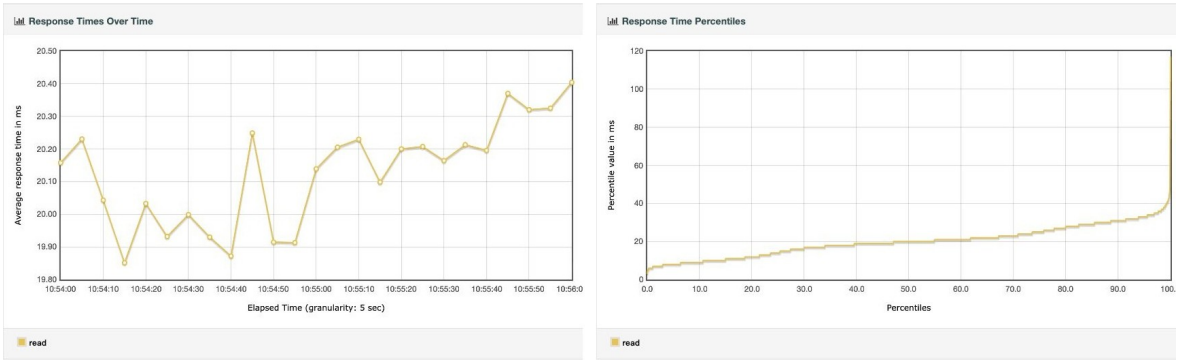


图 14　MongoDB 查询 25 线程的趋势与百分位分布

该档用于展示查询的性能转折附近状态。MySQL 与 MongoDB 各自的已测峰值线程数见前表；两图采用相同并发，便于直接比较。

查询 200 线程的 JMeter 曲线

200 线程下，MySQL 的 P50/P95/P99 中位数为 158.0/305.0/444.0 ms；MongoDB 为 170.0/258.0/328.0 ms。下图为该档第 1 次重复的 JMeter 报告页面裁切图，左侧为 5 秒窗口的响应时间趋势，右侧为完整正式窗口的百分位分布。

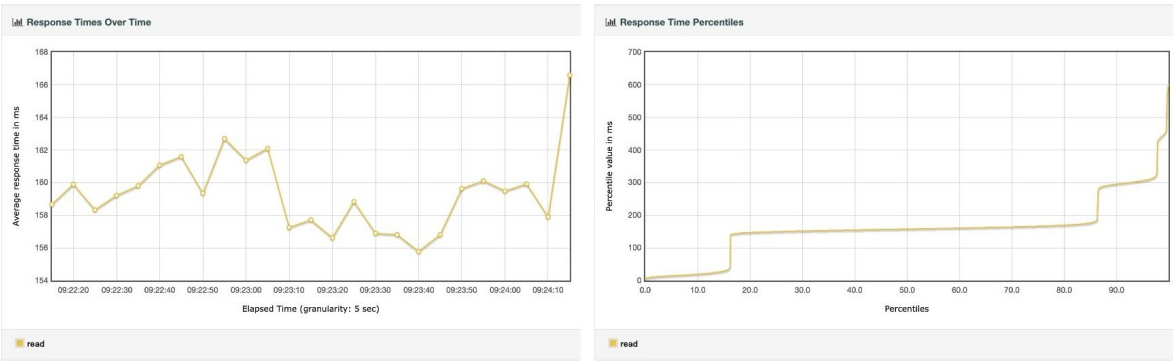


图 15　MySQL 查询 200 线程的趋势与百分位分布

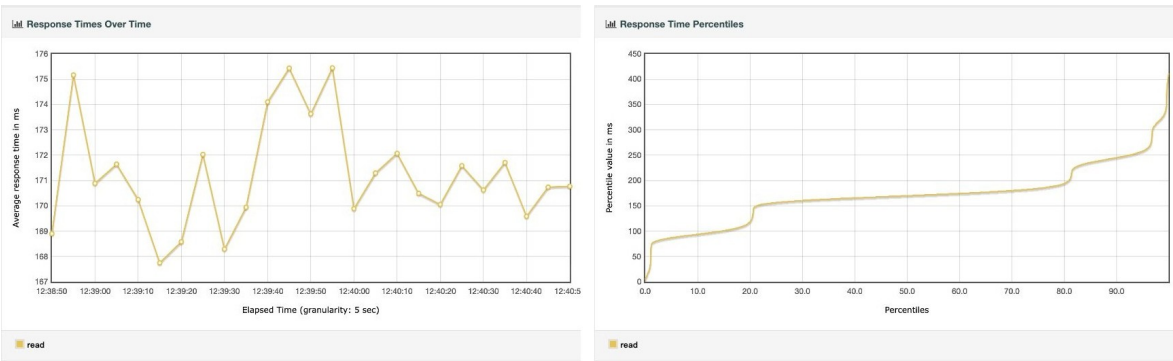


图 16　MongoDB 查询 200 线程的趋势与百分位分布

200 线程时，MySQL 的吞吐量为 1,258.0 次/秒、P95 为 305.0 ms；MongoDB 的吞吐量为 1,167.8 次/秒、P95 为 258.0 ms。200 线程为本次最高测试档位，三次测量均完成且错误率为 0%。

查询压力下的资源表现

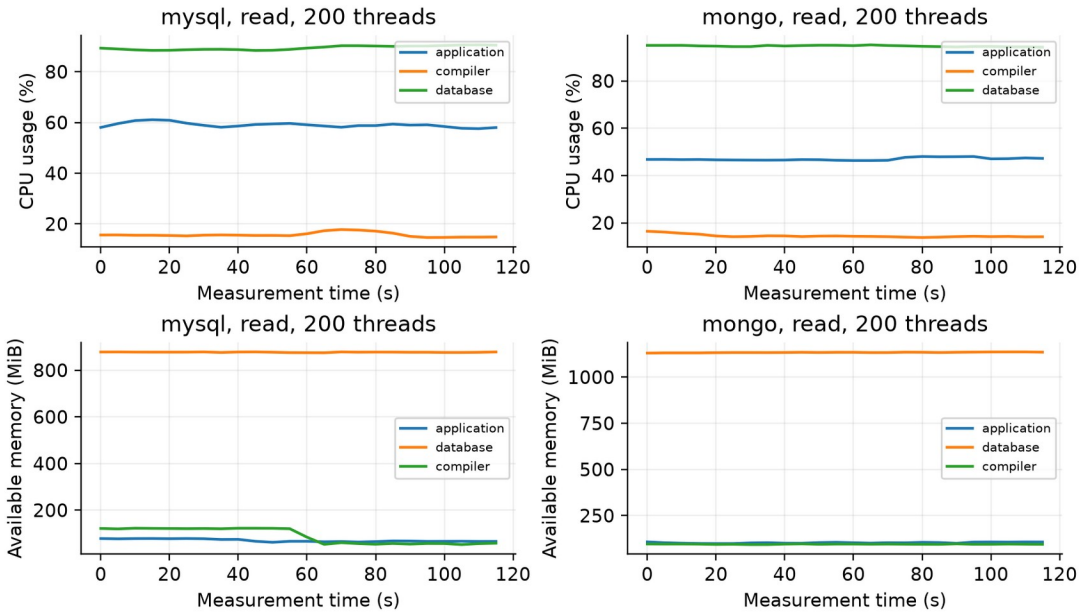


图 17　查询 200 线程第 1 次重复的主机 CPU 与可用内存

按压测机、应用机、数据库机顺序，平均 CPU 使用率为：MySQL 为 15.7%/59.0%/89.4%（压测机最小可用内存 52.4 MiB）；MongoDB 为 14.5%/47.0%/94.8%（压测机最小可用内存 92.8 MiB）。资源序列与延迟曲线使用同一正式窗口。

CPU 占用最高的环节分别为 MySQL 的数据库机（89.4%）、MongoDB 的数据库机（94.8%）。两库查询的数据库 CPU 均高于应用机和压测机，与吞吐平台及延迟增长同时出现。

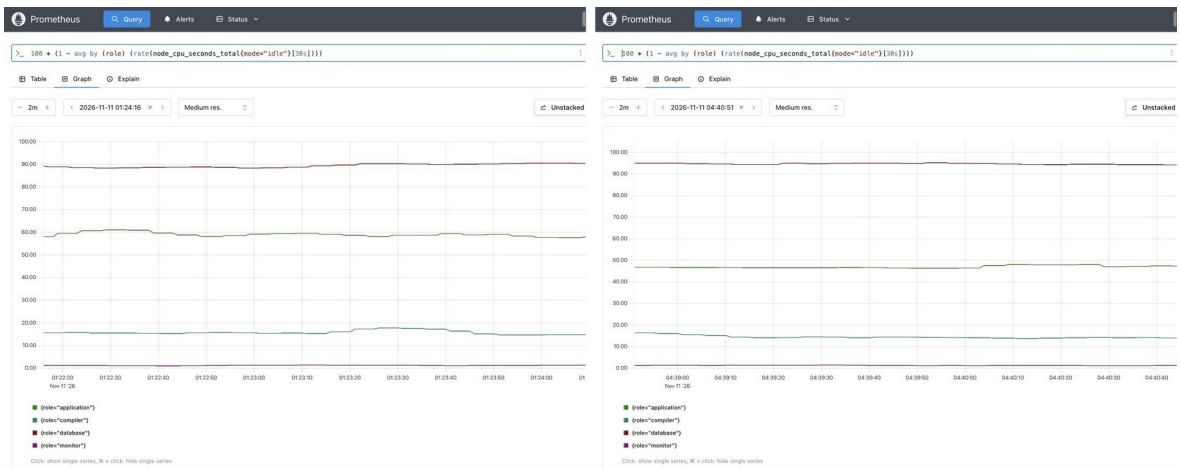


图 18　查询 200 线程正式窗口的 Prometheus CPU 页面，左 MySQL，右 MongoDB

5.2　写入阶梯升压

表 6　写入逐级压力测试性能

线程	MySQL 次/秒	P95 ms	P99 ms	MongoDB 次/秒	P95 ms	P99 ms
1	608.6	2.0	3.0	689.3	3.0	3.0
10	3909.9	4.0	5.0	3214.2	4.0	6.0
25	4556.2	10.0	14.0	4612.8	8.0	11.0
50	5136.9	22.0	32.0	4475.3	18.0	23.0
100	4117.8	63.0	108.0	4342.3	44.0	61.0
200	4608.5	123.0	200.0	4236.2	94.0	135.0

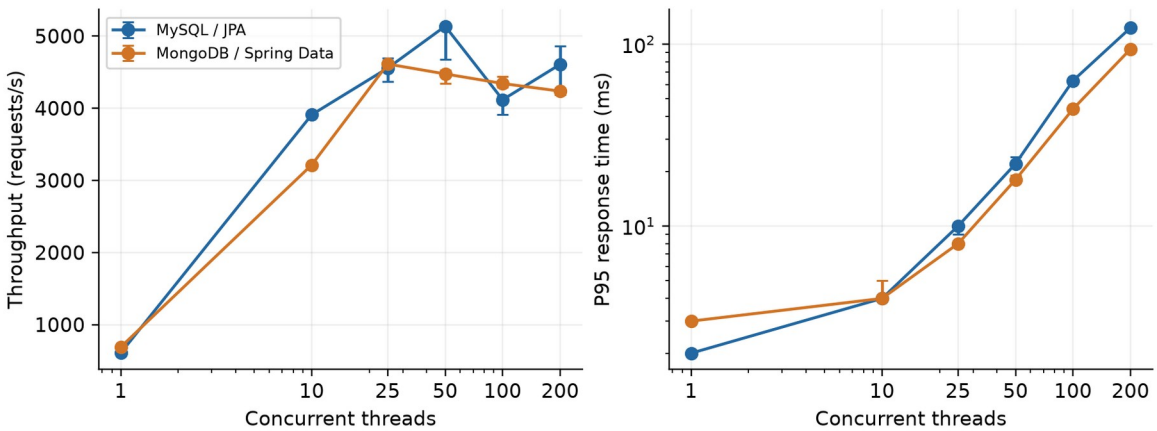


图 19　写入吞吐量及 P95 随并发变化，误差线为 3 次测量范围

在已测档位中，MySQL 最高中位吞吐量为 5,136.9 次/秒，对应 50 线程、P95 为 22.0 ms；MongoDB 最高中位吞吐量为 4,612.8 次/秒，对应 25 线程、P95 为 8.0 ms。

MySQL 从 50 线程增加到 200 线程，吞吐量较已测峰值下降 10.3%，P95 由 22.0 ms 上升至 123.0 ms，约为 5.6 倍。此区间继续增加并发的主要代价是延迟增长。

MongoDB 从 25 线程增加到 200 线程，吞吐量较已测峰值下降 8.2%，P95 由 8.0 ms 上升至 94.0 ms，约为 11.8 倍。此区间继续增加并发的主要代价是延迟增长。

在 200 线程档，MongoDB 相对 MySQL 的吞吐量低 8.1%、P95 低 23.6%。

写入 50 线程的 JMeter 曲线

50 线程下，MySQL 的 P50/P95/P99 中位数为 8.0/22.0/32.0 ms；MongoDB 为 11.0/18.0/23.0 ms。下图为该档第 1 次重复的 JMeter 报告页面裁切图，左侧为 5 秒窗口的响应时间趋势，右侧为完整正式窗口的百分位分布。

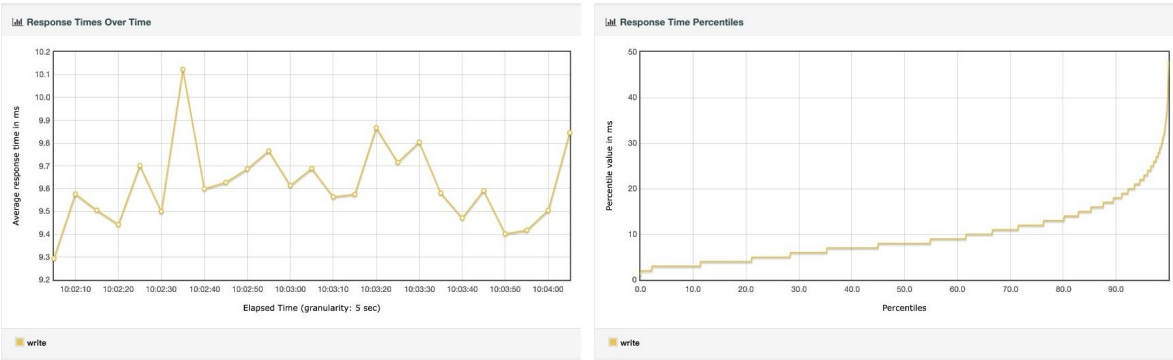


图 20　MySQL 写入 50 线程的趋势与百分位分布

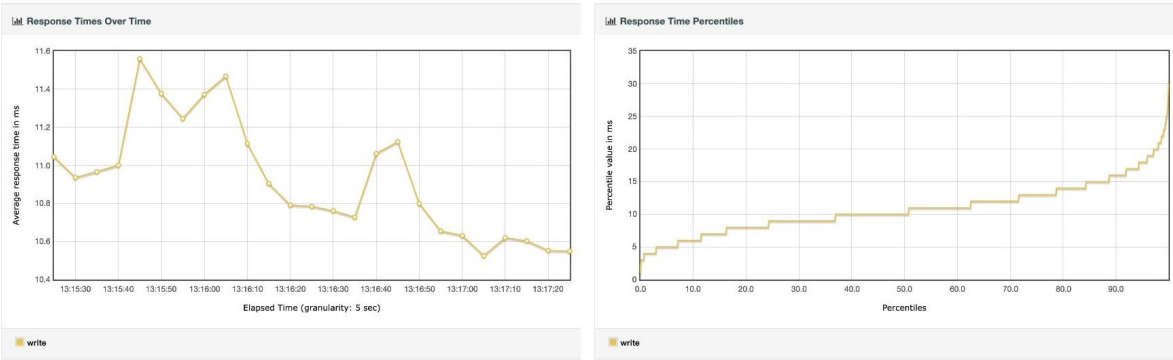


图 21　MongoDB 写入 50 线程的趋势与百分位分布

该档用于展示写入的性能转折附近状态。MySQL 与 MongoDB 各自的已测峰值线程数见前表；两图采用相同并发，便于直接比较。

写入 200 线程的 JMeter 曲线

200 线程下，MySQL 的 P50/P95/P99 中位数为 33.0/123.0/200.0 ms；MongoDB 为 46.0/94.0/135.0 ms。下图为该档第 1 次重复的 JMeter 报告页面裁切图，左侧为 5 秒窗口的响应时间趋势，右侧为完整正式窗口的百分位分布。

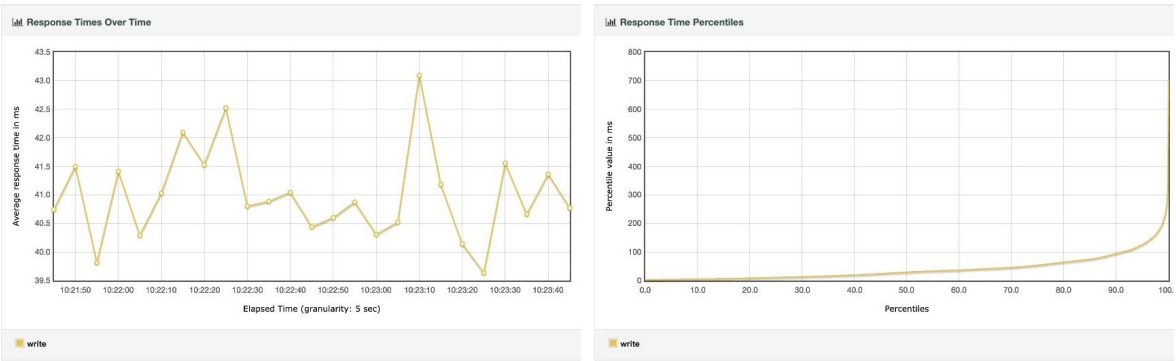


图 22　MySQL 写入 200 线程的趋势与百分位分布

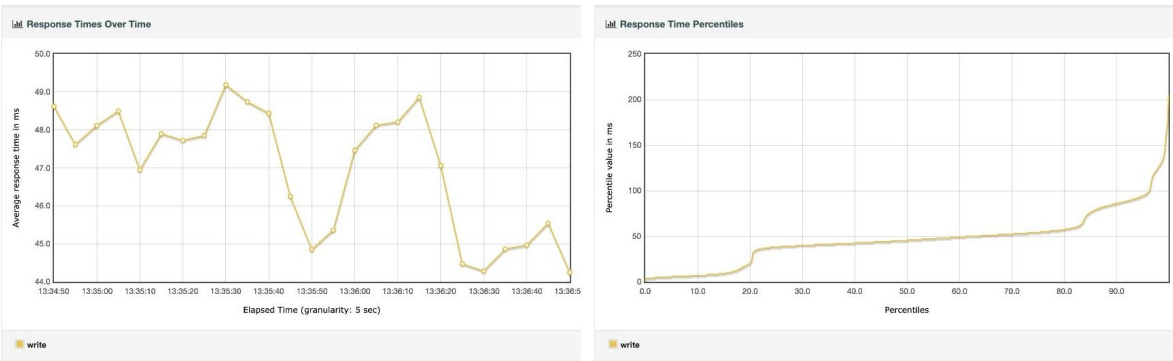


图 23　MongoDB 写入 200 线程的趋势与百分位分布

200 线程时，MySQL 的吞吐量为 4,608.5 次/秒、P95 为 123.0 ms；MongoDB 的吞吐量为 4,236.2 次/秒、P95 为 94.0 ms。200 线程为本次最高测试档位，三次测量均完成且错误率为 0%。

写入压力下的资源表现

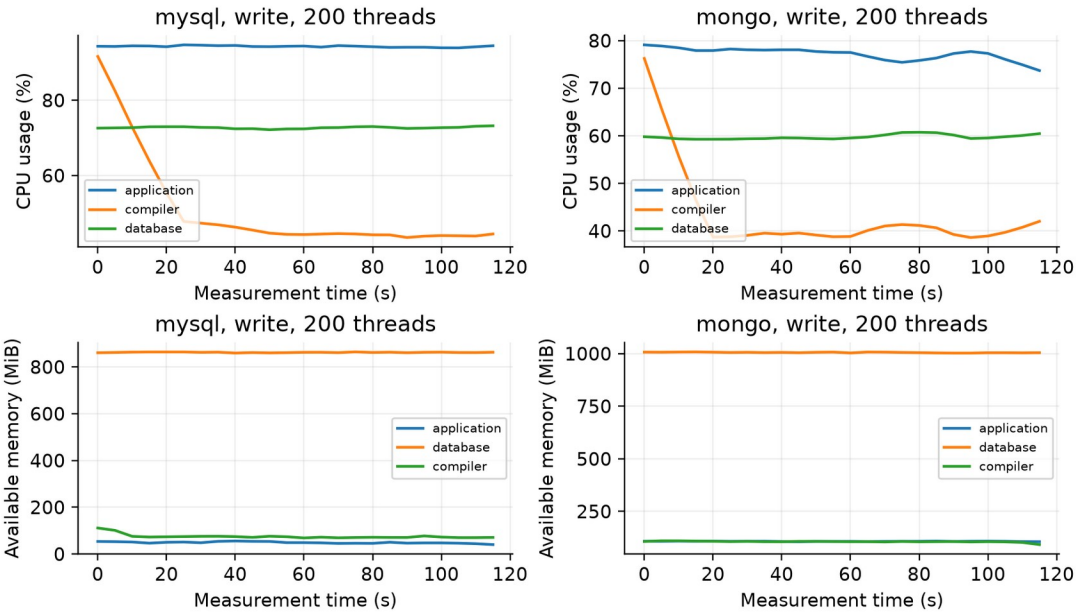


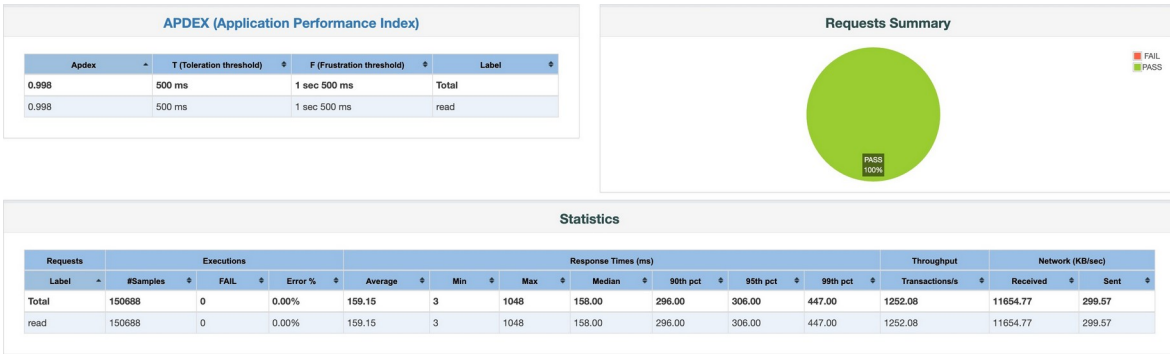
图 24　写入 200 线程第 1 次重复的主机 CPU 与可用内存

按压测机、应用机、数据库机顺序，平均 CPU 使用率为：MySQL 为 50.8%/94.3%/72.7%（压测机最小可用内存 68.9 MiB）；MongoDB 为 43.3%/77.2%/59.8%（压测机最小可用内存 92.4 MiB）。资源序列与延迟曲线使用同一正式窗口。

CPU 占用最高的环节分别为 MySQL 的应用机（94.3%）、MongoDB 的应用机（77.2%）。查询的高资源占用集中在数据库机，写入集中在应用机。该结果定位了当前链路的资源压力，具体等待原因还需结合数据库与应用剖析。

5.3　查询的 JMeter 原生汇总

下图展示 200 线程第 1 次重复的 APDEX 表、Requests Summary 饼图和 Statistics 表，均裁切自 JMeter 原生报告页面。低负载和转折附近的同类原图同时保存在 screenshots 目录。APDEX 阈值为 T=500 ms、F=1500 ms；饼图的 PASS 表示 HTTP 请求与断言通过，响应时间分布由 APDEX 及百分位指标描述。



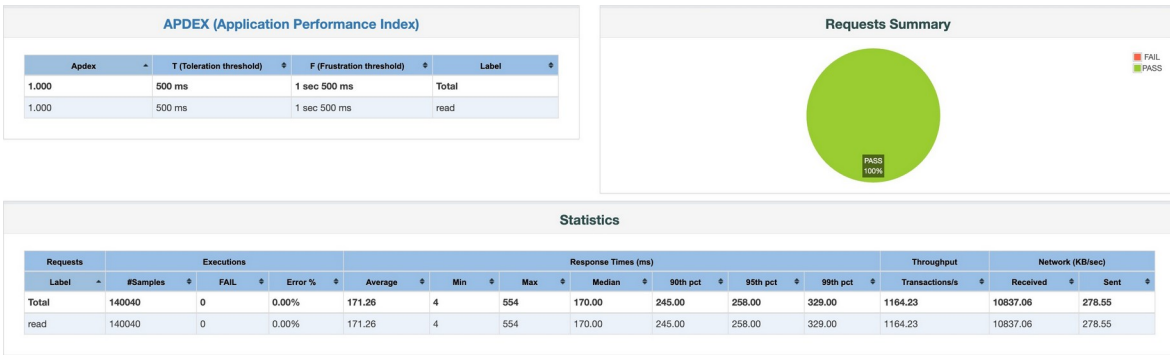
APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.998	500 ms	1 sec 500 ms	Total
0.998	500 ms	1 sec 500 ms	read

Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	150688	0	0.00%	159.15	3	1048	158.00	296.00	306.00	447.00	1252.08	11654.77	299.57
read	150688	0	0.00%	159.15	3	1048	158.00	296.00	306.00	447.00	1252.08	11654.77	299.57

图 25　MySQL 查询 200 线程第 1 次重复的 APDEX、请求饼图与统计表



APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	500 ms	1 sec 500 ms	Total
1.000	500 ms	1 sec 500 ms	read

Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	140040	0	0.00%	171.26	4	554	170.00	245.00	258.00	329.00	1164.23	10837.06	278.55
read	140040	0	0.00%	171.26	4	554	170.00	245.00	258.00	329.00	1164.23	10837.06	278.55

图 26　MongoDB 查询 200 线程第 1 次重复的 APDEX、请求饼图与统计表

Statistics 使用单次运行样本及其实际起止时刻计算吞吐量；正文对照表采用固定 120 秒窗口的每次请求数，再取 3 次中位数。两种统计口径对应的原始样本与重复编号均可在仓库中核对。

5.4　写入的 JMeter 原生汇总

下图展示 200 线程第 1 次重复的 APDEX 表、Requests Summary 饼图和 Statistics 表，均裁切自 JMeter 原生报告页面。低负载和转折附近的同类原图同时保存在 screenshots 目录。APDEX 阈值为 T=500 ms、F=1500 ms；饼图的 PASS 表示 HTTP 请求与断言通过，响应时间分布由 APDEX 及百分位指标描述。

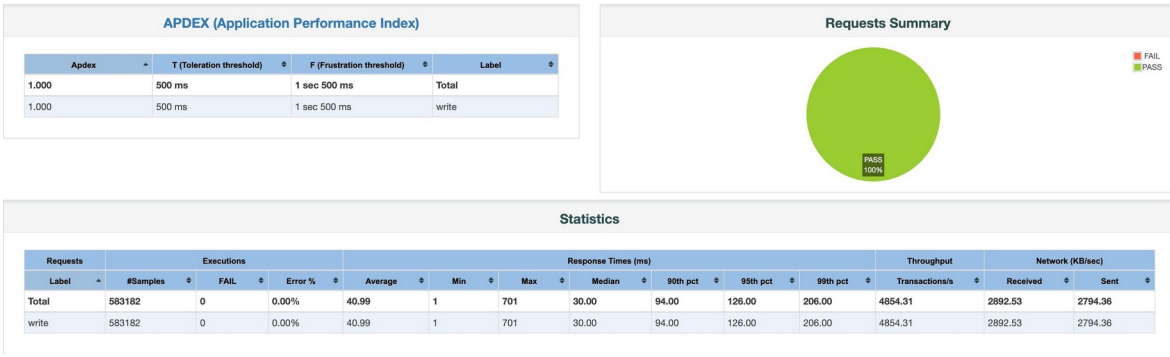


图 27　MySQL 写入 200 线程第 1 次重复的 APDEX、请求饼图与统计表

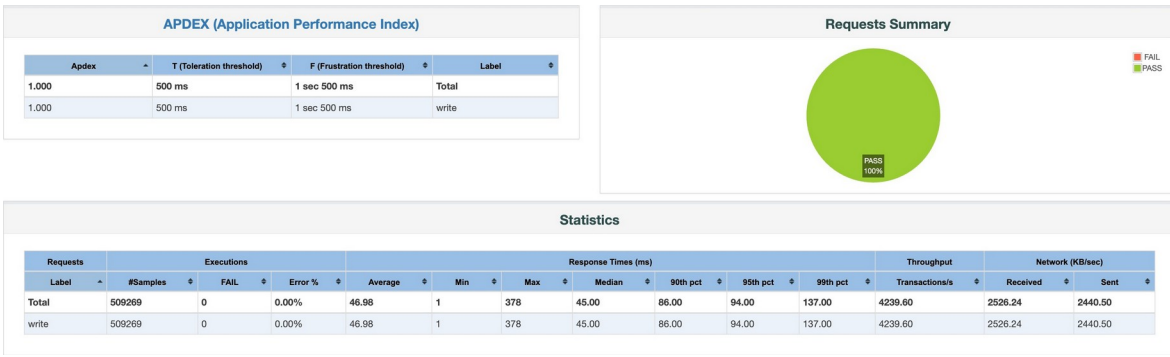


图 28　MongoDB 写入 200 线程第 1 次重复的 APDEX、请求饼图与统计表

Statistics 使用单次运行样本及其实际起止时刻计算吞吐量；正文对照表采用固定 120 秒窗口的每次请求数，再取 3 次中位数。两种统计口径对应的原始样本与重复编号均可在仓库中核对。

6　总结

本实验完成了同一日志模型和接口下的 JPA/MySQL 与 Spring Data MongoDB/MongoDB 对照实现，并在四台云主机上通过两轮部署完成 72 次正式测量。200 线程时，MySQL 的吞吐量为 1,258.0 次/秒、P95 为 305.0 ms；MongoDB 的吞吐量为 1,167.8 次/秒、P95 为 258.0 ms。200 线程时，MySQL 的吞吐量为 4,608.5 次/秒、P95 为 123.0 ms；MongoDB 的吞吐量为 4,236.2 次/秒、P95 为 94.0 ms。这些结果描述了本实验硬件、查询选择率、索引、连接池和写入确认设置下的应用链路表现。

查询返回 20 条数据并计算 2497 条匹配记录的总数，写入采用单条持久化并等待确认；两类操作承担不同的数据库工作。并发增加后的性能变化应结合吞吐量、P95/P99 和主机资源判断。本文保留重复测量范围、原始错误和资源序列，使读写差异及性能转折均可追溯到具体运行。

表 7　实验完成与数据核验

核验项目	结果
正式测量	72 次，24 个实现与操作并发组合，各重复 3 次
正式请求数	20,730,914
正式错误数	0
记录数核对	全部运行均完成核对
原始数据	升压预热、正式窗口、收尾 JTL 无损压缩及 SHA-256
程序与材料	https://github.com/Baakarshan/mysql-mongodb-log-lab4

参考文献

[1] Spring. Spring Data JPA Reference Documentation. https://docs.spring.io/spring-data/jpa/reference/.

[2] Spring. Spring Data MongoDB Reference Documentation. https://docs.spring.io/spring-data/mongodb/reference/.

[3] Apache Software Foundation. Apache JMeter User Manual: Generating Dashboard Report. https://jmeter.apache.org/usermanual/generating-dashboard.html.

[4] MongoDB. Write Concern. https://www.mongodb.com/docs/v7.0/reference/write-concern/.

[5] Oracle. MySQL 8.4 Reference Manual: InnoDB Startup Options and System Variables. https://dev.mysql.com/doc/refman/8.4/en/innodb-parameters.html.